

PROPLEDGER ARCHITECTURE SERIES — VOLUME 09 OF 10

Frontend Architecture & React 19

TypeScript, TanStack Query, Tailwind CSS & Enterprise UI Kit

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume establishes the frontend engineering standards of PropLedger. Built with React 19, TypeScript 5.6, Vite 6, and Tailwind CSS 3.4, it provides an exhaustive technical analysis of modern component architectures, server-state management via TanStack React Query v5, custom design system primitives, and 25 real-world technical interview questions.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	React 19 Core Paradigms	Actions, async transitions, ref as prop, and Fiber reconciliation
03	Enterprise TypeScript Architecture	Strict typings, generics envelopes, and discriminated unions
04	State Management Taxonomy	Server state vs global client state vs local UI state breakdown
05	TanStack React Query v5	Stale-while-revalidate, automatic deduplication, and cache invalidation
06	Atomic Component Architecture	Custom UI kit: StatCard, Modal, StatusBadge, Pagination, SearchBar
07	Responsive Layout & Protected Routes	Sidebar navigation, TopNav, and client-side authentication guards
08	Tailwind CSS & Design Systems	Utility-first design tokens, dark-slate palette, and zero-runtime CSS
09	Interactive Financial Charts	Recharts SVG visualizations: Area, Bar, and Pie distribution charts
10	Top 25 Frontend Interview Questions	High-yield React, hooks, performance, and Web Vitals interview Q&A

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedger production repository.

2. REACT 19 CORE PARADIGMS & FIBER RECONCILIATION

MODERN FUNCTIONAL COMPONENTS, ACTIONS & VIRTUAL DOM

1. Key Architectural Additions in React 19

- 1. Actions & Async Transitions: Built-in handling of asynchronous mutations via `useActionState` and `useOptimistic`, eliminating manual loading state flags.
- 2. `ref` as a Standard Prop: Function components accept `ref` directly as a prop; `forwardRef` is officially deprecated.
- 3. Native Document Metadata: Built-in support for `<title>` and `<meta>` tags directly inside components with automated hoisting to document `<head>`.
- 4. React Compiler Readiness: Prepares the codebase for automated memoization, phasing out manual `useMemo`/`useCallback` boilerplate.

2. The Fiber Reconciliation Engine

React's Fiber architecture divides rendering into two distinct phases:

- Render Phase (Asynchronous): Builds Virtual DOM tree, computes diffs using an $O(N)$ heuristic algorithm, and can be paused or aborted.
- Commit Phase (Synchronous): Applies calculated mutations to the actual browser DOM in a single atomic flush, preventing layout thrashing.

3. ENTERPRISE TYPESCRIPT ARCHITECTURE

STRICT TYPINGS, GENERICS & DISCRIMINATED UNIONS

1. Type Parity Between Frontend and Backend

To eliminate runtime payload mismatch bugs, PropLedger's TypeScript interfaces (src/types/index.ts) mirror Java DTOs 1:1:

```
// CODE LISTING: TYPESCRIPT 5.6 – DOMAIN INTERFACES & DISCRIMINATED UNIONS

// propledger-frontend/src/types/index.ts
export interface Invoice {
  id: number;
  invoiceNumber: string;
  leaseId: number;
  invoiceDate: string;
  dueDate: string;
  totalAmount: number;
  balanceDue: number;
  status: InvoiceStatus;
  items?: InvoiceItem[];
}

// Discriminated String Union Types:
export type InvoiceStatus = 'PENDING' | 'PARTIALLY_PAID' | 'PAID' | 'OVERDUE' | 'VOID';
export type LeaseStatus = 'DRAFT' | 'ACTIVE' | 'TERMINATED' | 'EXPIRED' | 'RENEWED';
export type PriorityLevel = 'LOW' | 'MEDIUM' | 'HIGH' | 'EMERGENCY';
```

2. Generic API Response Envelopes

PropLedger wraps all network calls in generic interfaces: 'ApiResponse<T>' and 'PagedResponse<T>', providing compile-time auto-completion down to individual entity attributes.

4. STATE MANAGEMENT TAXONOMY: WHY NOT REDUX?

SERVER STATE VS GLOBAL CLIENT STATE VS EPHEMERAL UI STATE

1. The Three State Categories in PropLedger

State Category	Management Tool	Data Examples	Key Operational Features
Server State	TanStack React Query v5	Invoices, Leases, Units, Metrics	Stale-while-revalidate, auto-caching, ded...
Global Client State	React Context API	Auth Token, User Profile, Theme	Low-frequency updates, accessible acro...
Local UI State	useState / useReducer	Modal Open/Close, Form Inputs	Ephemeral, isolated to individual compon...

2. Why Redux is an Anti-Pattern in Modern Real Estate SPAs

In enterprise SaaS platforms, 90% of data is owned by the server. Redux forces hundreds of lines of boilerplate (actions, reducers, thunks, selectors, normalization) simply to mirror server data. TanStack Query manages server state natively in 5 lines of declarative code.

5. TANSTACK REACT QUERY V5 & CACHE INVALIDATION

STALE-WHILE-REVALIDATE CACHING & COORDINATED INVALIDATION

1. The Coordinated Invalidation Pattern

When a user records a payment, four independent UI widgets must update simultaneously: the invoice grid, the payments history list, the dashboard revenue charts, and the aging AR breakdown:

```
// CODE LISTING: TYPESCRIPT – TANSTACK QUERY INVALIDATION PATTERN

// PaymentsPage.tsx: Multi-Cache Invalidation on Mutation
const queryClient = useQueryClient();

const paymentMutation = useMutation({
  mutationFn: (paymentData: PaymentRequest) => api.post('/api/payments', paymentData),
  onSuccess: () => {
    // Atomically invalidate and trigger background refetch of all dependent queries:
    queryClient.invalidateQueries({ queryKey: ['payments'] });
    queryClient.invalidateQueries({ queryKey: ['invoices'] });
    queryClient.invalidateQueries({ queryKey: ['dashboard-stats'] });
    queryClient.invalidateQueries({ queryKey: ['aging-ar'] });
  },
  onError: (err: any) => {
    toast.error(err.response?.data?.message || 'Payment recording failed');
  }
});
```

2. Stale-While-Revalidate Settings

Queries are configured with 'staleTime: 60000' (1 minute). Navigating between pages reuses cached data instantly with zero layout shifts, while refreshing in the background.

6. ATOMIC COMPONENT ARCHITECTURE & UI PRIMITIVES

CUSTOM ENTERPRISE DESIGN SYSTEM PRIMITIVES

1. Custom UI Kit Inventory (src/components/ui/)

Component	Key Props	Design Functionality
StatCard	title, value, icon, change, trend	Executive metric card with trending color accents.
Modal	isOpen, onClose, title, children	Accessible dialog with backdrop blur, portal rendering.
StatusBadge	status (string)	Semantic color pill (PAID = emerald, OVERDUE = rose, PENDING = am...
Pagination	currentPage, totalPages, onPageChange	0-indexed page controls with active page indicators.
SearchBar	value, onChange, placeholder	Debounced search input with clearing action.

2. Production Code: StatusBadge Component

```
// CODE LISTING: TSX – STATUSBADGE DESIGN TOKEN IMPLEMENTATION

// src/components/ui/index.tsx: StatusBadge Component
export const StatusBadge = ({ status }: { status: string }) => {
  const styles: Record<string, string> = {
    ACTIVE: 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20',
    PAID: 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20',
    OCCUPIED: 'bg-blue-500/10 text-blue-400 border-blue-500/20',
    PENDING: 'bg-amber-500/10 text-amber-400 border-amber-500/20',
    OVERDUE: 'bg-rose-500/10 text-rose-400 border-rose-500/20',
    MAINTENANCE: 'bg-purple-500/10 text-purple-400 border-purple-500/20',
  };
  return (
    <span className={`px-2.5 py-0.5 rounded-full text-xs font-semibold border ${styles[status]} || 'bg-slate-800 text...`
      {status}
    </span>
  );
};
```

7. RESPONSIVE LAYOUT & CLIENT ROUTE GUARDS

COLLAPSIBLE SIDEBAR, NAVIGATION & REACT ROUTER V6

1. Client Layout Architecture

PropLedger implements a responsive dashboard layout with persistent left navigation, top bar controls, and protected route guards:

// CODE LISTING: TSX – PROTECTED ROUTE GUARD IMPLEMENTATION

```
// App.tsx: Protected Route Architecture with React Router v6
const ProtectedRoute = ({ children, allowedRoles }: { children: ReactNode; allowedRoles?: string[] }) => {
  const { user, token } = useAuth();

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  if (allowedRoles && !allowedRoles.some(role => user?.roles.includes(role))) {
    return <Navigate to="/dashboard" replace />;
  }

  return <DashboardLayout>{children}</DashboardLayout>;
};
```

2. Navigation Hierarchy

The navigation sidebar links to all 13 core enterprise modules: Dashboard, Properties, Units, Tenants, Leases, Invoices, Payments, Expenses, Vendors, Maintenance, Reports, and Audit Logs.

8. TAILWIND CSS & ENTERPRISE DESIGN TOKENS

ZERO-RUNTIME CSS, PURGING & DARK-SLATE AESTHETICS

1. Why Tailwind CSS Over Runtime CSS-in-JS

Runtime CSS-in-JS (like styled-components) parses CSS in JavaScript on every render, adding CPU and memory overhead. Tailwind CSS generates static utility classes at build time, resulting in **zero runtime overhead** and sub-50KB production CSS bundles.

2. PropLedger Enterprise Design Palette

Token Category	Tailwind Class	HEX Code	Design Role
Background Dark	bg-slate-950	#020617	Global application canvas background
Surface Card	bg-slate-900/50	#0f172a	Glassmorphic card panels and modals
Primary Accent	text-emerald-500	#10b981	KPI stat gains, paid badges, submit buttons
Secondary Accent	text-sky-500	#0ea5e9	Links, active tabs, unit selections
Border Accent	border-slate-800	#1e293b	Clean structural divider lines

9. INTERACTIVE FINANCIAL DATA VISUALIZATIONS

RECHARTS DECLARATIVE SVG CURVES, TOOLTIPS & AREA CHARTS

1. Recharts Architecture in PropLedger

PropLedger leverages Recharts to render high-performance, responsive SVG charts without canvas pixelation:

// CODE LISTING: TSX — RECHARTS AREACHART IMPLEMENTATION

```
// RevenueChart.tsx: Monthly Revenue vs OpEx Curves
export const RevenueChart = ({ data }: { data: MonthlyFinancial[] }) => (
  <ResponsiveContainer width="100%" height={300}>
    <AreaChart data={data}>
      <defs>
        <linearGradient id="colorRev" x1="0" y1="0" x2="0" y2="1">
          <stop offset="5%" stopColor="#10b981" stopOpacity={0.4}/>
          <stop offset="95%" stopColor="#10b981" stopOpacity={0}/>
        </linearGradient>
      </defs>
      <XAxis dataKey="month" stroke="#64748b" />
      <YAxis stroke="#64748b" tickFormatter={(v) => `$$${v / 1000}k`} />
      <Tooltip contentStyle={{ backgroundColor: '#0f172a', borderColor: '#334155' }} />
      <Area type="monotone" dataKey="revenue" stroke="#10b981" fill="url(#colorRev)" />
      <Area type="monotone" dataKey="expenses" stroke="#f43f5e" fill="none" />
    </AreaChart>
  </ResponsiveContainer>
);
```

10. TOP 25 FRONTEND INTERVIEW QUESTIONS & ANSWERS

SENIOR REACT & TYPESCRIPT ENGINEERING SCENARIOS

1. High-Yield Frontend Interview Questions

Question 1: Explain the difference between useMemo, useCallback, and React.memo.

Answer: React.memo is a higher-order component memoizing component render output based on prop shallow equality. useMemo caches the computed result of an expensive calculation between renders. useCallback caches a function definition to preserve reference equality when passing callbacks to memoized child components.

Question 2: Why is using array index as a key prop in lists an anti-pattern?

Answer: React uses keys to match virtual DOM nodes across renders. If array indexes are used and items are sorted, filtered, or prepended, indexes shift, causing React to associate existing DOM nodes with different data items. This corrupts form inputs, breaks CSS animations, and causes unnecessary re-renders.

Question 3: How do you prevent race conditions in rapid search inputs?

Answer: 1) Debounce search input changes by 300ms. 2) In TanStack Query or Axios, pass an AbortController signal to the fetcher function. When the user types a new character, previous pending HTTP requests are aborted at the browser network layer, guaranteeing that stale responses never overwrite fresh data.

FRONTEND INTERVIEW KEY TAKEAWAY

Articulating server-state management via React Query versus global client state proves you can build production SPAs without unnecessary Redux overhead.